

MSI ANALYSIS

✓ MSI Change Detection Report (Tri-temporal)

Triplets: E425_1 and E425_15**Files:** BEFORE / ON / AFTER MSI GeoTIFFs

This notebook performs:

1. Data loading + sanity checks
2. Band visualization (RGB + false color)
3. NDVI + NDWI computation
4. Tri-temporal change detection:
 - Index differencing (Δ NDVI, Δ NDWI)
 - Multi-spectral Change Vector Analysis (CVA magnitude)
 - Persistent vs Transient change separation
5. Thresholding (Otsu) + morphological cleanup
6. Export change masks + index rasters as GeoTIFF and PNG

```
!pip -q install rasterio scikit-image

import os, re, glob
import numpy as np
import rasterio
from rasterio.enums import Resampling
import matplotlib.pyplot as plt

from skimage.filters import threshold_otsu
from skimage.morphology import remove_small_objects, remove_small_holes, disk, opening, closing
```

✓ 1) Locate MSI triplets

We auto-detect files named like:

- BEFORE_E425_1_msi.tif
- ON_E425_1_msi.tif
- AFTER_E425_1_msi.tif and same for 15.

```
ROOT = "/content"
paths = glob.glob(os.path.join(ROOT, "**", "*E425*msi.tif"), recursive=True)

print("Found MSI files:", len(paths))
for p in sorted(paths):
    print(p)

def build_triplets(paths):
```

```

pat = re.compile(r"(BEFORE|ON|AFTER)_E425_(\d+)_msi\.tif$", re.IGNORECASE)
trip = {}
for p in paths:
    name = os.path.basename(p)
    m = pat.search(name)
    if not m:
        continue
    phase = m.group(1).upper()
    tid = m.group(2)
    trip.setdefault(tid, {})
    trip[tid][phase] = p
return trip

triplets = build_triplets(paths)

print("\nTriplets detected:", sorted(triplets.keys()))
for tid in sorted(triplets.keys()):
    print(tid, triplets[tid])

assert "1" in triplets and "15" in triplets, "Could not find both triplets 1 and 15. Check filenames."
for tid in ["1", "15"]:
    for k in ["BEFORE", "ON", "AFTER"]:
        assert k in triplets[tid], f"Missing {k} for triplet {tid}"

Found MSI files: 6
/content/AFTER_E425_15_msi.tif
/content/AFTER_E425_1_msi.tif
/content/BEFORE_E425_15_msi.tif
/content/BEFORE_E425_1_msi.tif
/content/ON_E425_15_msi.tif
/content/ON_E425_1_msi.tif

Triplets detected: ['1', '15']
1 {'AFTER': '/content/AFTER_E425_1_msi.tif', 'BEFORE': '/content/BEFORE_E425_1_msi.tif', 'ON': '/content/ON_E425_1_msi.tif'}
15 {'BEFORE': '/content/BEFORE_E425_15_msi.tif', 'ON': '/content/ON_E425_15_msi.tif', 'AFTER': '/content/AFTER_E425_15_msi.tif'}

```

2) Band mapping

We use Sentinel-2 style band indices:

- Blue = 2
- Green = 3
- Red = 4
- NIR = 8 Optional:
- SWIR1 = 11 (if present)
- SWIR2 = 12 (if present)

If your TIFF has different band ordering, update this dictionary.

```

# Sentinel-2 typical indices (1-based)
BANDS = {
    "BLUE": 2,
    "GREEN": 3,
    "RED": 4,
    "NIR": 8,

```

```

"SWIR1": 11, # optional
"SWIR2": 12, # optional
}

def safe_band(stack, b1):
    return stack[b1-1] if b1 <= stack.shape[0] else None

def to_reflectance_like(x):
    # many exports are 0..10000 scaled int
    return x/10000.0 if np.nanmax(x) > 1.5 else x

def read_stack(path, plot_max=1200):
    with rasterio.open(path) as src:
        meta = src.profile.copy()
        nodata = src.nodata
        full = src.read().astype(np.float32)

        h, w = src.height, src.width
        scale = min(plot_max/h, plot_max/w, 1.0)
        oh, ow = int(h*scale), int(w*scale)
        small = src.read(out_shape=(src.count, oh, ow), resampling=Resampling.bilinear).astype(np.float32)
    return full, small, meta, nodata

def fixed_rgb(small, r=4,g=3,b=2, minv=0.0, maxv=0.30, gamma=1.2):
    R,G,B = safe_band(small,r), safe_band(small,g), safe_band(small,b)
    rgb = np.stack([R,G,B], axis=-1)
    rgb = to_reflectance_like(rgb)
    rgb = np.clip((rgb-minv)/(maxv-minv), 0, 1)
    rgb = rgb ** (1/gamma)
    return rgb

def show_grid(imgs, titles, nrows, ncols, figsize=(14,8), cmap=None, vmin=None, vmax=None):
    fig, axes = plt.subplots(nrows, ncols, figsize=figsize)
    axes = np.array(axes).reshape(-1)
    for i, ax in enumerate(axes):
        ax.axis("off")
        if i >= len(imgs):
            continue
        im = ax.imshow(imgs[i], cmap=cmap, vmin=vmin, vmax=vmax)
        ax.set_title(titles[i], fontsize=10)
    plt.tight_layout()
    plt.show()

```

```

def panel_rgb_falsecolor(tid):
    imgs, titles = [], []
    for phase in ["BEFORE", "ON", "AFTER"]:
        _, small, _, _ = read_stack(triplets[tid][phase])
        small = to_reflectance_like(small)

        rgb = fixed_rgb(small, r=4,g=3,b=2)
        fcc = fixed_rgb(small, r=8,g=4,b=3) # NIR-Red-Green

        imgs += [rgb, fcc]
        titles += [f"{tid} | {phase} | RGB (4,3,2)", f"{tid} | {phase} | False (8,4,3)"]
    show_grid(imgs, titles, nrows=3, ncols=2, figsize=(12,14))

```

```
for tid in ["1","15"]:  
    panel_rgb_falsecolor(tid)
```


1 | BEFORE | RGB (4,3,2)



1 | BEFORE | False (8,4,3)



1 | ON | RGB (4,3,2)



1 | ON | False (8,4,3)



1 | AFTER | RGB (4,3,2)

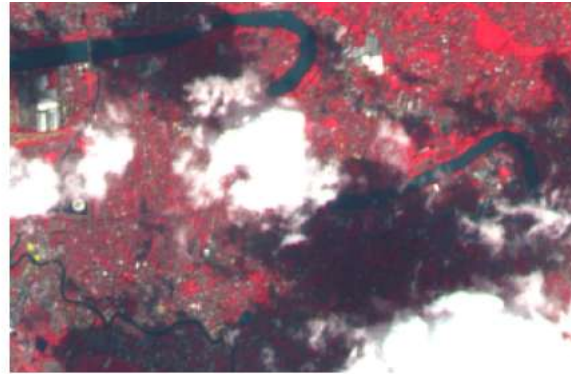


1 | AFTER | False (8,4,3)





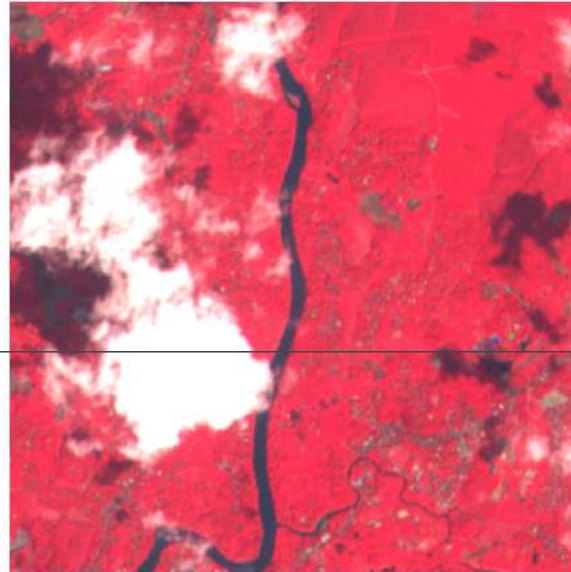
15 | BEFORE | RGB (4,3,2)



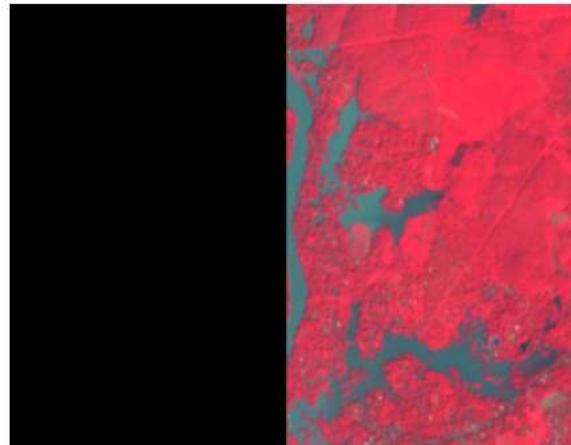
15 | BEFORE | False (8,4,3)



15 | ON | RGB (4,3,2)



15 | ON | False (8,4,3)



3) Sanity check

We print band count, shape, dtype, nodata for BEFORE/ON/AFTER in each triplet.

```
def print_info(path):
    with rasterio.open(path) as src:
        print(os.path.basename(path))
        print(" bands:", src.count, "size:", (src.height, src.width),
              "dtype:", src.dtypes[0], "nodata:", src.nodata, "crs:", src.crs)

for tid in sorted(triplets.keys()):
    print("\n=== Triplet", tid, "===")
    for k in ["BEFORE", "ON", "AFTER"]:
        print_info(triplets[tid][k])
```

```
=== Triplet 1 ===
BEFORE_E425_1_msi.tif
bands: 13 size: (316, 316) dtype: uint16 nodata: None crs: EPSG:32644
ON_E425_1_msi.tif
bands: 13 size: (316, 316) dtype: uint16 nodata: None crs: EPSG:32644
AFTER_E425_1_msi.tif
bands: 13 size: (316, 316) dtype: uint16 nodata: None crs: EPSG:32644

=== Triplet 15 ===
BEFORE_E425_15_msi.tif
bands: 13 size: (316, 316) dtype: uint16 nodata: None crs: EPSG:32644
ON_E425_15_msi.tif
bands: 13 size: (316, 316) dtype: uint16 nodata: None crs: EPSG:32644
AFTER_E425_15_msi.tif
bands: 13 size: (316, 316) dtype: uint16 nodata: None crs: EPSG:32644
```

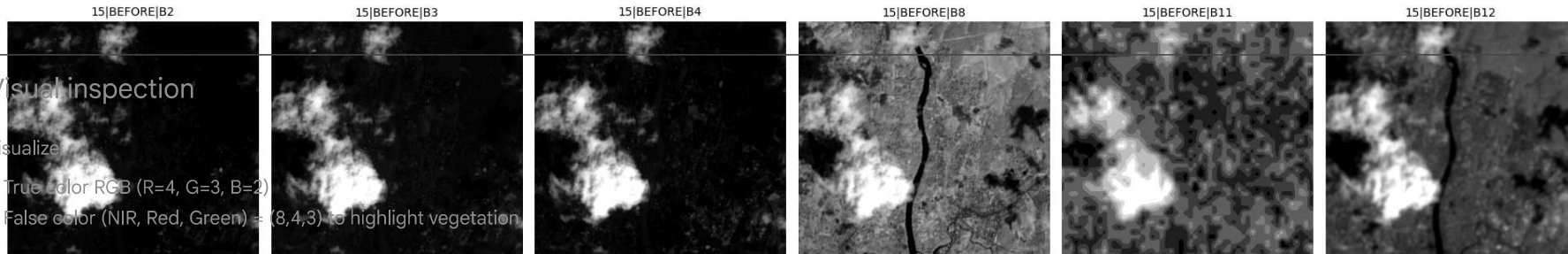
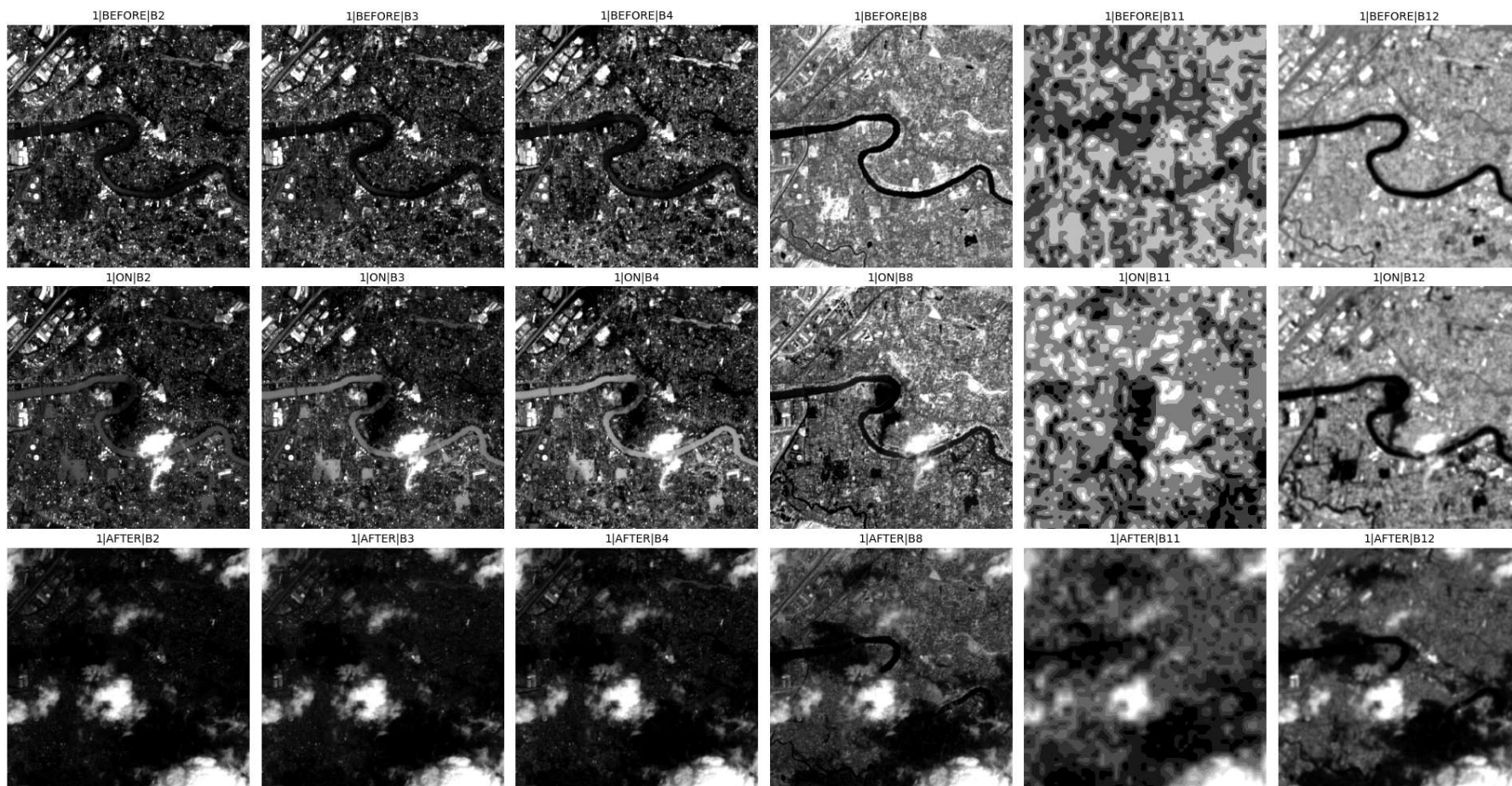
```
def stretch01(x, lo=2, hi=98):
    x = x.astype(np.float32)
    a = np.nanpercentile(x, lo)
    b = np.nanpercentile(x, hi)
    if b-a < 1e-6:
        return np.zeros_like(x)
    return np.clip((x-a)/(b-a), 0, 1)

def band_grid(tid):
    # decide which bands exist from BEFORE file
    full, small, _, _ = read_stack(triplets[tid]["BEFORE"])
    n = small.shape[0]
    band_list = [2,3,4,8]
    if n >= 11: band_list.append(11)
    if n >= 12: band_list.append(12)

    imgs, titles = [], []
    for phase in ["BEFORE", "ON", "AFTER"]:
        _, s, _, _ = read_stack(triplets[tid][phase])
        s = to_reflectance_like(s)
        for b in band_list:
            imgs.append(stretch01(safe_band(s,b)))
            titles.append(f"{tid}|{phase}|B{b}")
    show_grid(imgs, titles, nrows=3, ncols=len(band_list), figsize=(3.2*len(band_list), 10), cmap="gray")
```



```
for tid in ["1","15"]:  
    band_grid(tid)
```

4) Visual inspection

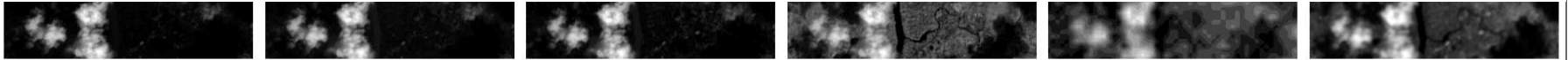
We visualize

- True color RGB (R=4, G=3, B=2)
- False color (NIR, Red, Green) = (8,4,3) to highlight vegetation

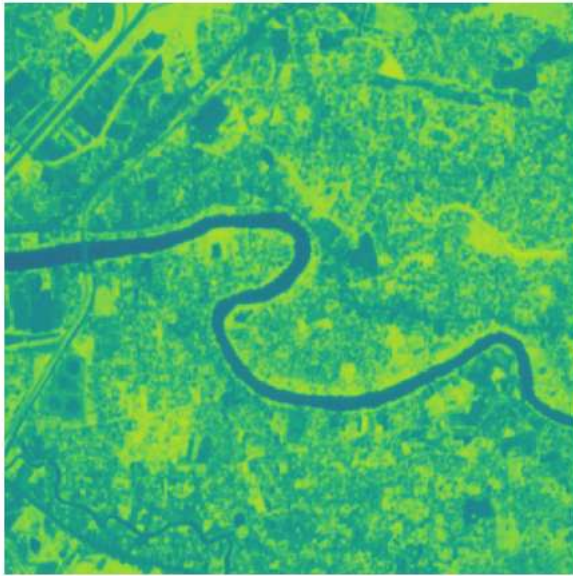
```
def ndvi(stack):
    red = safe_band(stack, BANDS["RED"])
    nir = safe_band(stack, BANDS["NIR"])
    return np.clip((nir-red)/(nir+red+1e-6), -1, 1)

def ndwi(stack):
    green = safe_band(stack, BANDS["GREEN"])
    nir = safe_band(stack, BANDS["NIR"])
    return np.clip((green-nir)/(green+nir+1e-6), -1, 1)
```

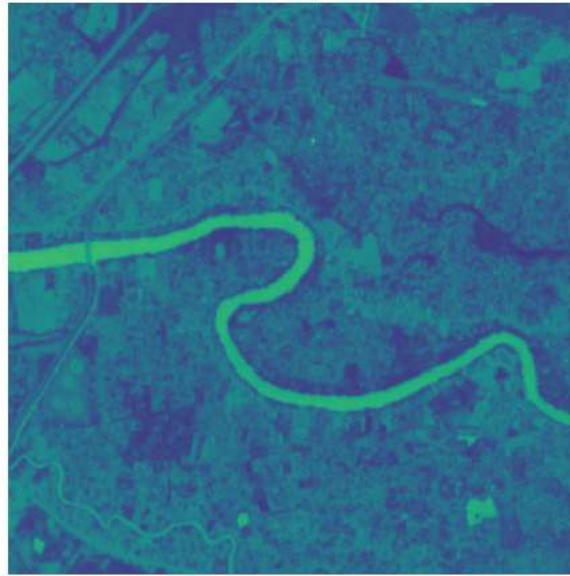
```
def panel_indices(tid):  
    imgs, titles = [], []  
    for phase in ["BEFORE", "ON", "AFTER"]:  
        _, small, _, _ = read_stack(triplets[tid][phase])  
        small = to_reflectance_like(small)  
        imgs += [ndvi(small), ndwi(small)]  
        titles += [f"{tid}|{phase}|NDVI", f"{tid}|{phase}|NDWI"]  
    show_grid(imgs, titles, nrow=3, ncol=2, figsize=(10,14), cmap=None, vmin=-1, vmax=1)  
  
for tid in ["1", "15"]:  
    panel_indices(tid)
```



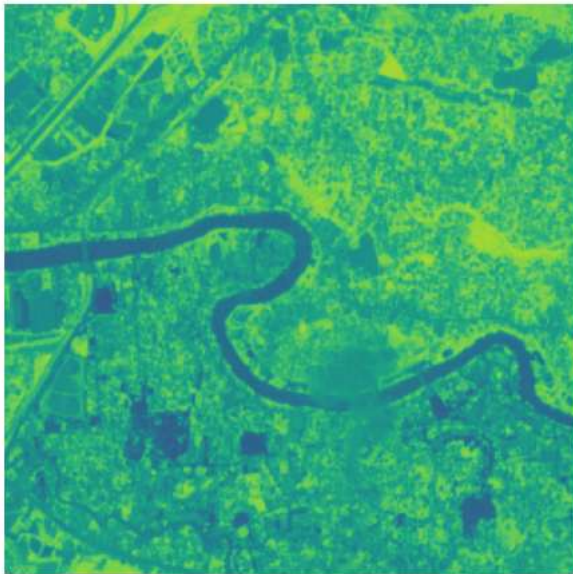
1|BEFORE|NDVI



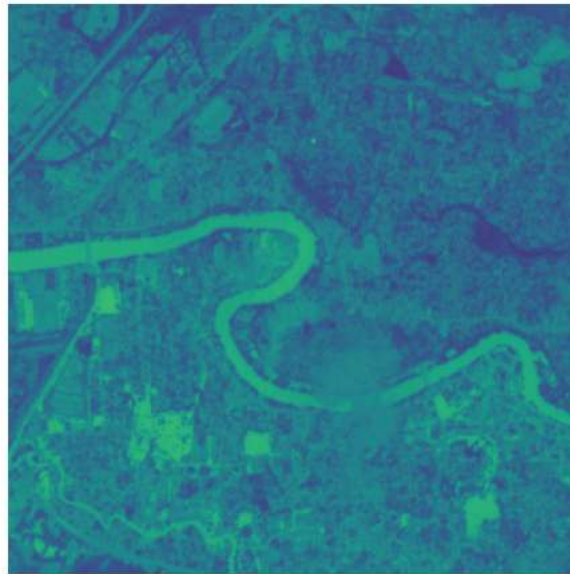
1|BEFORE|NDWI



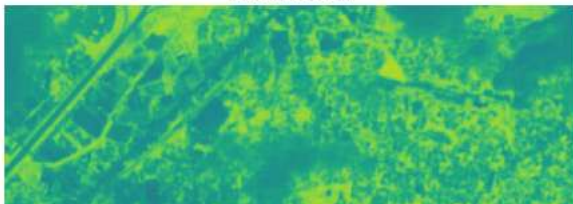
1|ON|NDVI



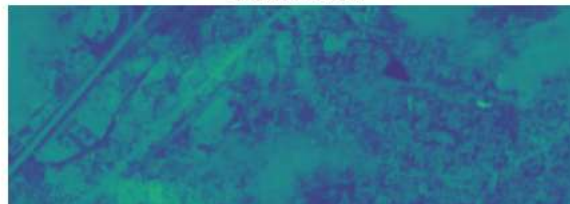
1|ON|NDWI

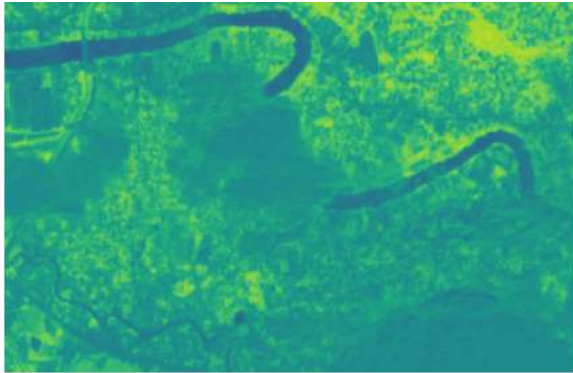


1|AFTER|NDVI

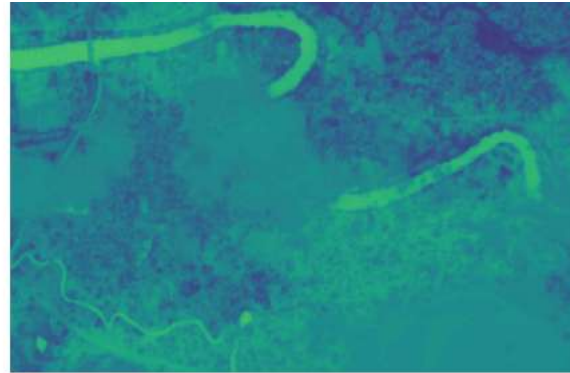


1|AFTER|NDWI

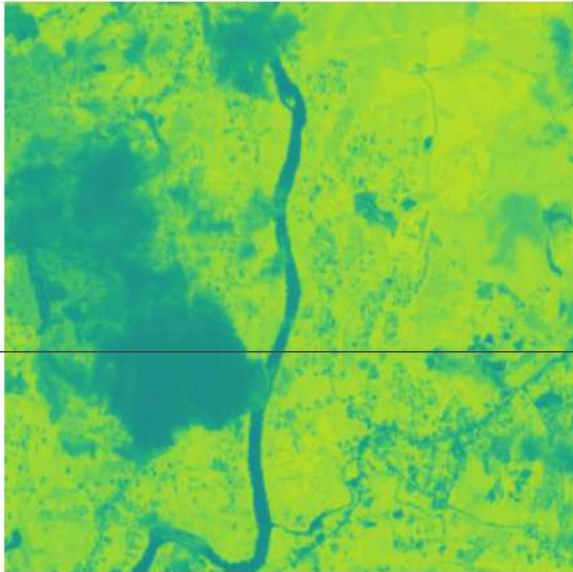




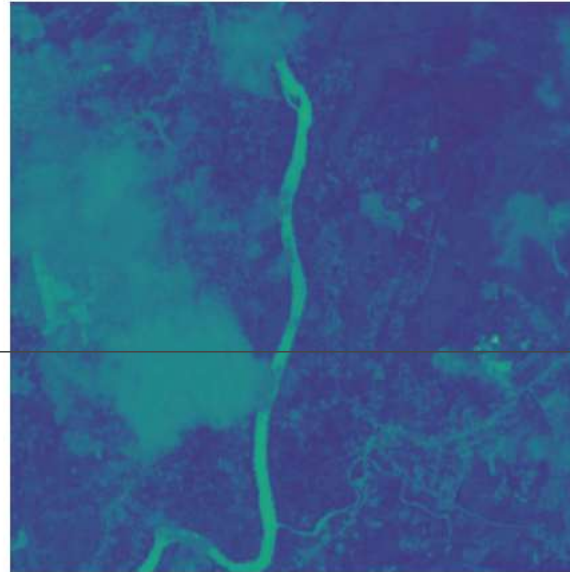
15|BEFORE|NDVI



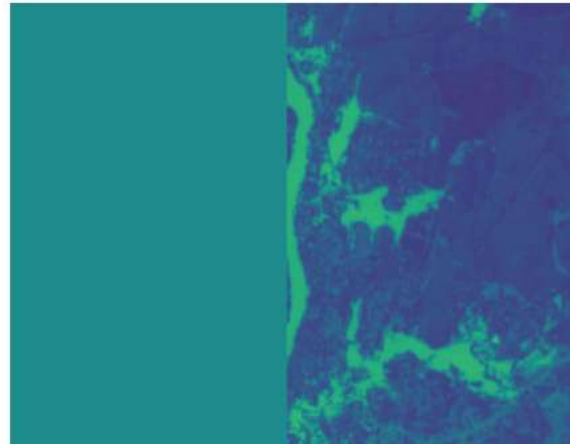
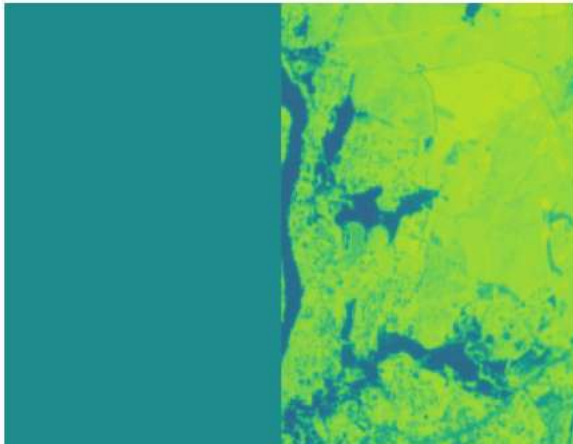
15|BEFORE|NDWI



15|ON|NDVI



15|ON|NDWI

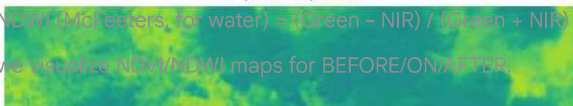
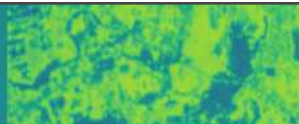


5) NDVI and NDWI

We compute:

- $NDVI = (NIR - Red) / (NIR + Red)$
- $NDWI \text{ (for fresh water or water)} = (Green - NIR) / (Green + NIR)$

Then we visualize NDVI/NDWI maps for BEFORE/ON/AFTER



```
def otsu_mask(change, min_abs=0.03):
    x = np.abs(change)
    x = x[np.isfinite(x)]
    x = x[x >= min_abs]
    if x.size < 200:
        return None, None
    th = threshold_otsu(x)
    return th, (np.abs(change) >= th)

def clean(mask, min_area=300):
    if mask is None:
        return None
    m = mask.copy()
    m = remove_small_holes(m, area_threshold=min_area)
    m = remove_small_objects(m, min_size=min_area)
    m = closing(m, disk(2))
    m = opening(m, disk(1))
    return m

def tri_masks(before_idx, on_idx, after_idx, min_abs=0.03, min_area=300):
    d_on = on_idx - before_idx
    d_af = after_idx - before_idx

    th_on, m_on = otsu_mask(d_on, min_abs=min_abs)
    th_af, m_af = otsu_mask(d_af, min_abs=min_abs)

    m_on = clean(m_on, min_area=min_area) if m_on is not None else None
    m_af = clean(m_af, min_area=min_area) if m_af is not None else None

    if m_on is None or m_af is None:
        return d_on, d_af, None, None, None, (th_on, th_af)
    persistent = m_on & m_af
    transient = m_on & (~m_af)
    return d_on, d_af, m_on, m_af, (persistent, transient), (th_on, th_af)

def panel_change_for_index(tid, idx_name="NDVI"):
    # compute on downsampled for visualization
    _, b, _, _ = read_stack(triplets[tid]["BEFORE"])
    _, o, _, _ = read_stack(triplets[tid]["ON"])
    _, a, _, _ = read_stack(triplets[tid]["AFTER"])
    b = to_reflectance_like(b); o = to_reflectance_like(o); a = to_reflectance_like(a)

    if idx_name.upper() == "NDVI":
        ib, io, ia = ndvi(b), ndvi(o), ndvi(a)
    else:
        ib, io, ia = ndwi(b), ndwi(o), ndwi(a)
```



```
d_on, d_af, m_on, m_af, pm, ths = tri_masks(ib, io, ia, min_abs=0.03, min_area=250)
print(f"Triplet {tid} | {idx_name} thresholds (ON, AFTER):", ths)

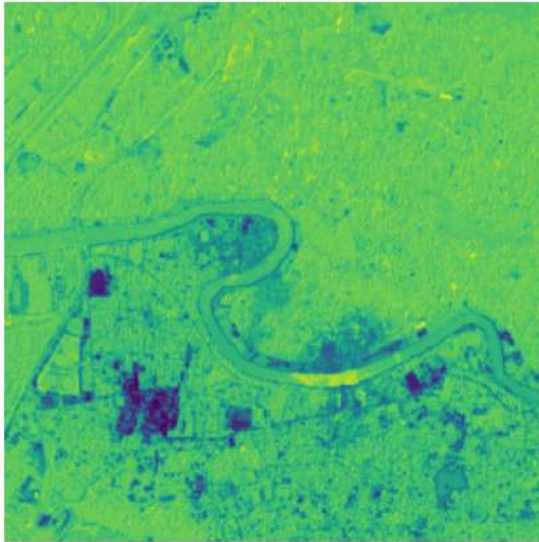
imgs = [d_on, d_af,
        (m_on.astype(float) if m_on is not None else np.zeros_like(d_on)),
        (m_af.astype(float) if m_af is not None else np.zeros_like(d_on))]
titles = [f"{tid}|{idx_name}|ΔON", f"{tid}|{idx_name}|ΔAFTER",
          f"{tid}|{idx_name}|Mask_ON", f"{tid}|{idx_name}|Mask_AFTER"]

if pm is not None:
    persistent, transient = pm
    imgs += [persistent.astype(float), transient.astype(float)]
    titles += [f"{tid}|{idx_name}|Persistent", f"{tid}|{idx_name}|Transient"]
    show_grid(imgs, titles, nrows=2, ncols=3, figsize=(14,9), vmin=None, vmax=None)
else:
    show_grid(imgs, titles, nrows=1, ncols=4, figsize=(16,4), vmin=None, vmax=None)

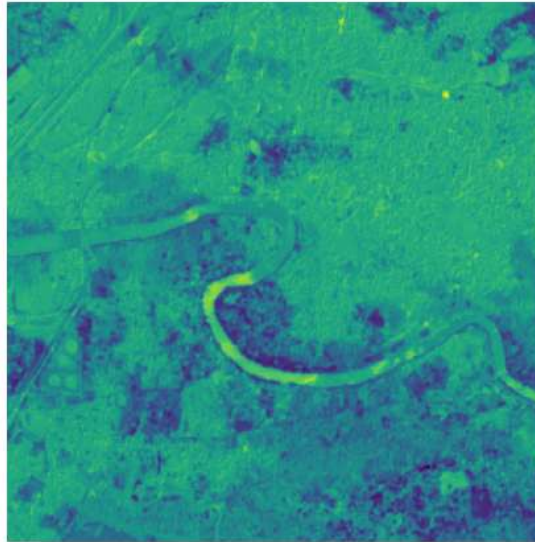
for tid in ["1", "15"]:
    panel_change_for_index(tid, "NDVI")
    panel_change_for_index(tid, "NDWI")
```


Triplet 1 | NDVI thresholds (ON, AFTER): (np.float32(0.3223071), np.float32(0.24178901))

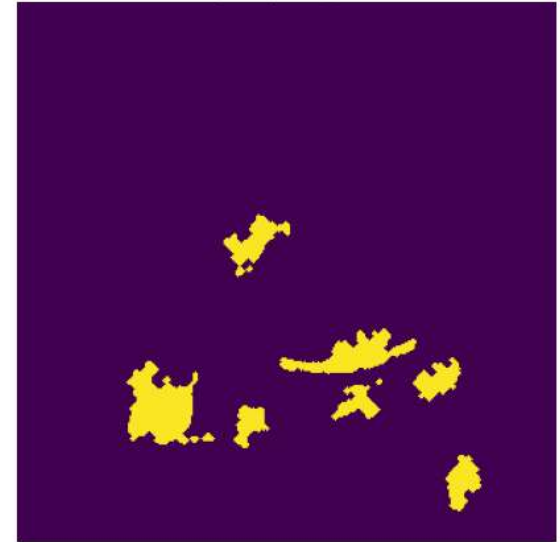
1|NDVI|ΔON



1|NDVI|ΔAFTER



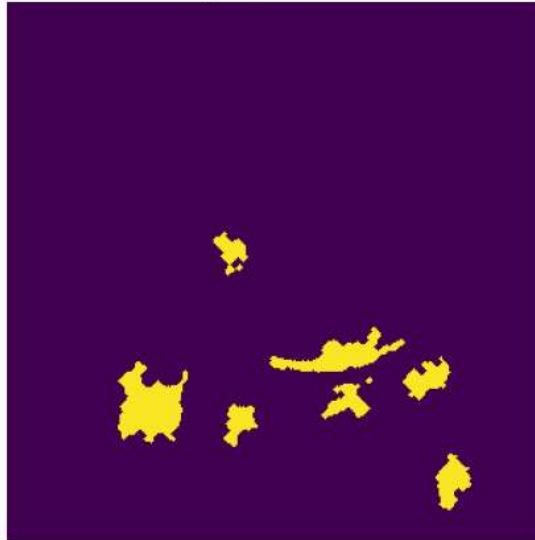
1|NDVI|Mask_ON



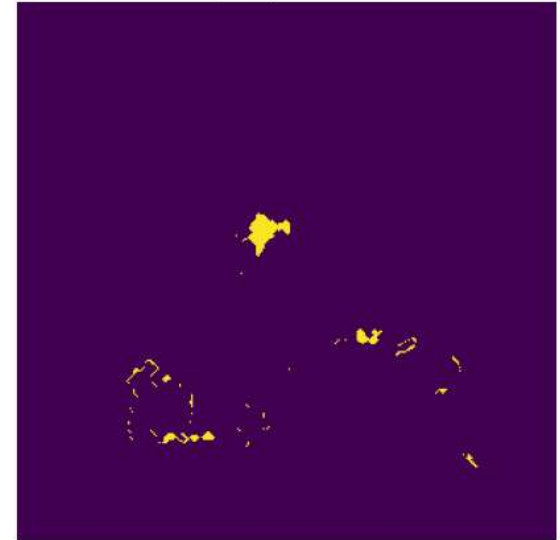
1|NDVI|Mask_AFTER



1|NDVI|Persistent

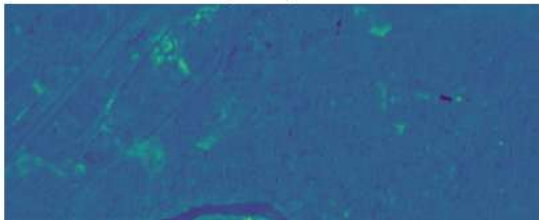


1|NDVI|Transient

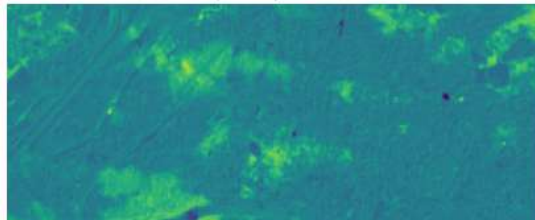


Triplet 1 | NDWI thresholds (ON, AFTER): (np.float32(0.2675923), np.float32(0.21511897))

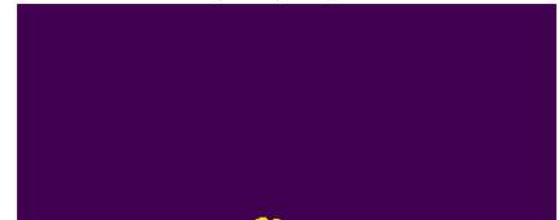
1|NDWI|ΔON

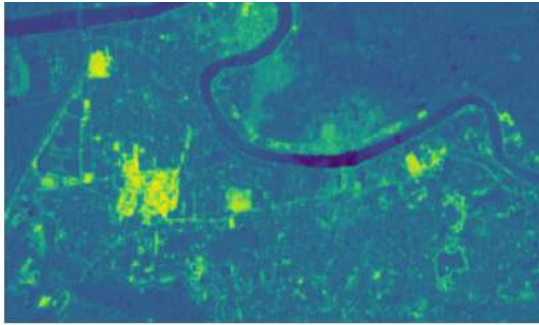


1|NDWI|ΔAFTER

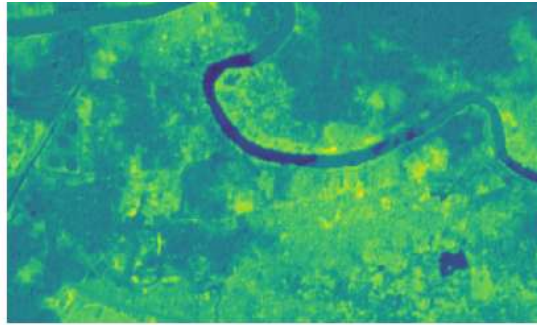


1|NDWI|Mask_ON

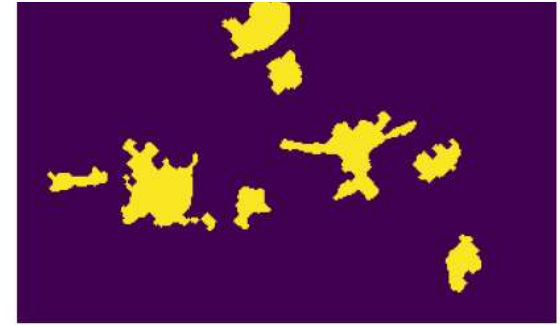




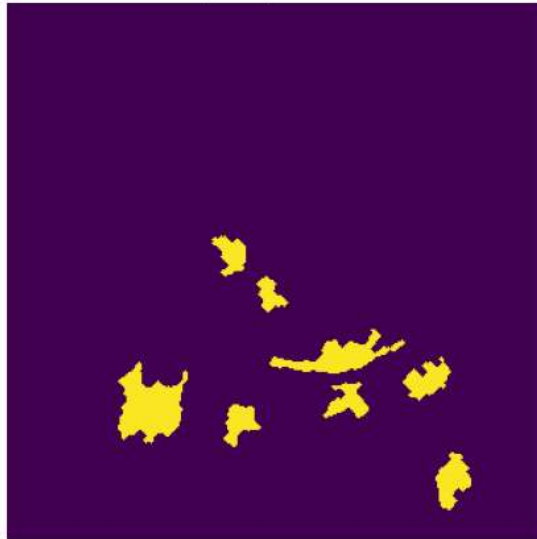
1|NDWI|Mask_AFTER



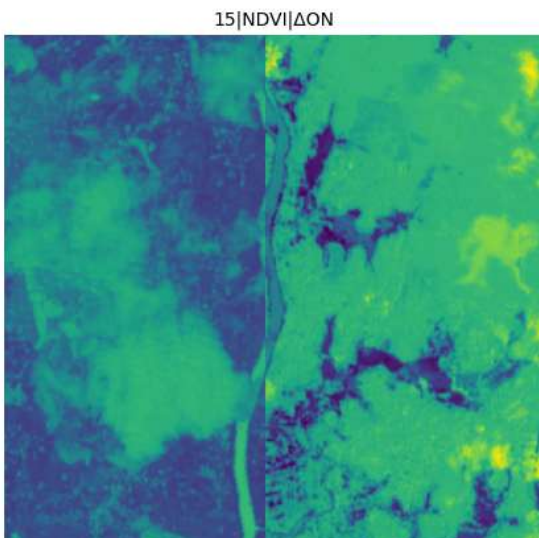
1|NDWI|Persistent



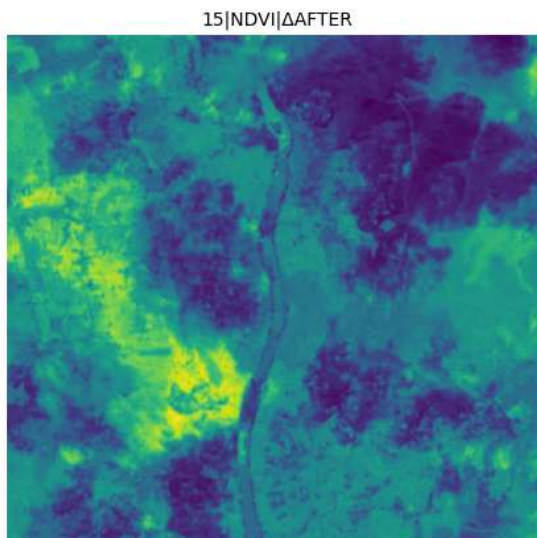
1|NDWI|Transient



Triplet 15 | NDVI thresholds (ON, AFTER): (np.float32(0.36971915), np.float32(0.3559786))



15|NDVI|ΔON



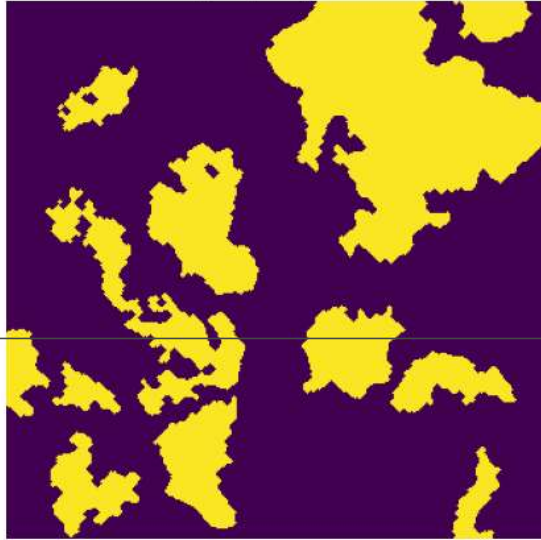
15|NDVI|ΔAFTER



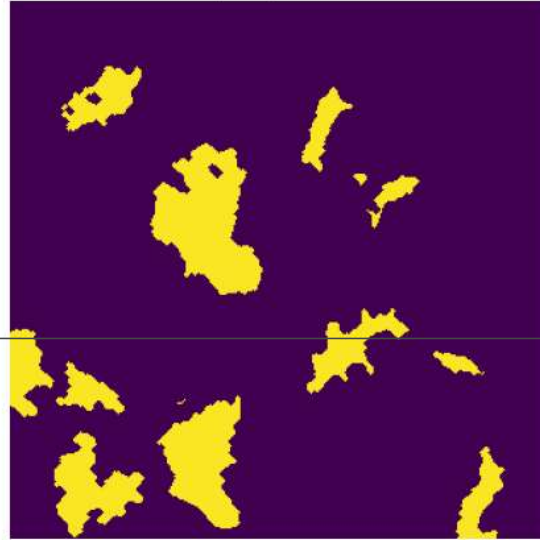
15|NDVI|Mask_ON



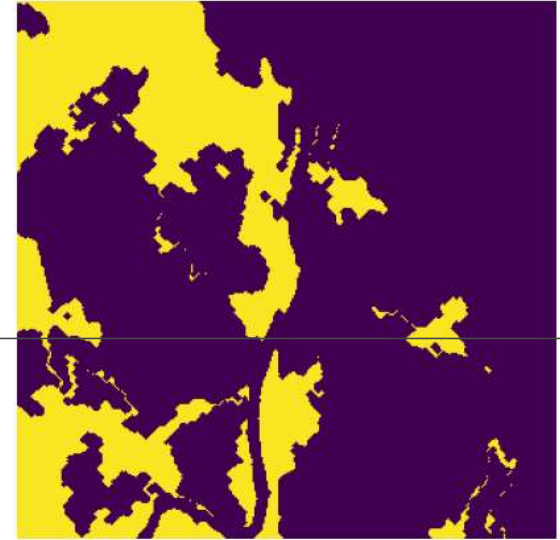
15|NDVI|Mask_AFTER



15|NDVI|Persistent

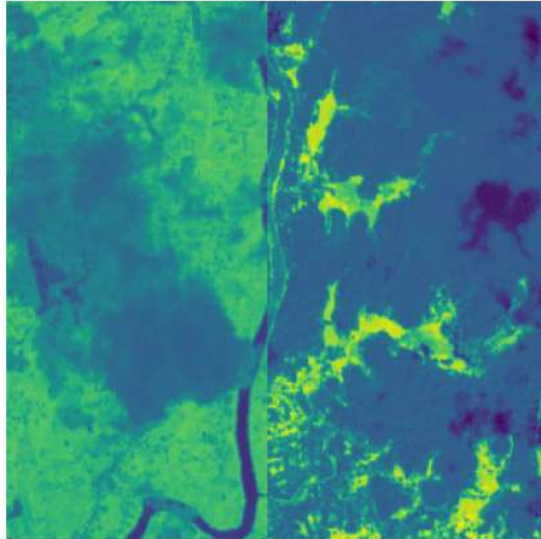


15|NDVI|Transient

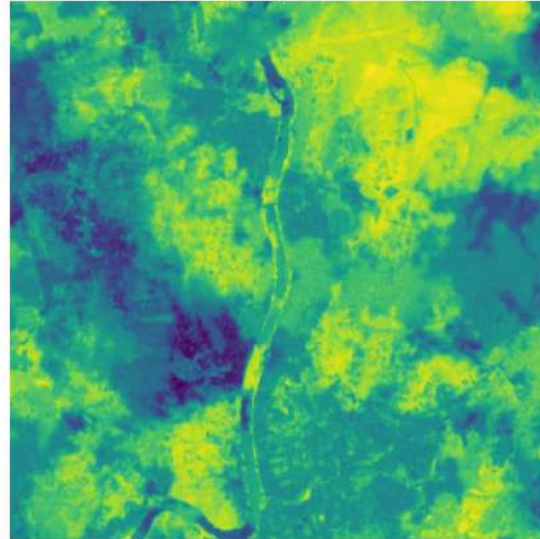


Triplet 15 | NDWI thresholds (ON, AFTER): (np.float32(0.32077527), np.float32(0.27250737))

15|NDWI|ΔON



15|NDWI|ΔAFTER



15|NDWI|Mask_ON



15|NDWI|Mask_AFTER



15|NDWI|Persistent



15|NDWI|Transient



6) Tri-temporal change detection (Index differencing)

We compute:

- $\Delta\text{NDVI}_{\text{on}} = \text{NDVI}(\text{ON}) - \text{NDVI}(\text{BEFORE})$
- $\Delta\text{NDVI}_{\text{after}} = \text{NDVI}(\text{AFTER}) - \text{NDVI}(\text{BEFORE})$ (similarly for NDWI)

Then we derive:

- **Persistent change:** change appears in ON and remains in AFTER
- **Transient change:** change appears in ON but disappears in AFTER

We do thresholding using **Otsu** on the absolute change magnitude.

```
def otsu_mask(change_map, min_abs=0.02):
    """Otsu threshold on |change| after removing tiny values."""
    x = np.abs(change_map)
    x = x[np.isfinite(x)]
    x = x[x >= min_abs]
    if x.size < 100:
        return None, None
    th = threshold_otsu(x)
    return th, (np.abs(change_map) >= th)

def cleanup_mask(mask, min_area=200):
    if mask is None:
        return None
    m = mask.copy()
    m = remove_small_holes(m, area_threshold=min_area)
    m = remove_small_objects(m, min_size=min_area)
    m = closing(m, disk(2))
    m = opening(m, disk(1))
    return m

def tri_temporal_masks(before_idx, on_idx, after_idx, min_abs=0.02, min_area=200):
    d_on = on_idx - before_idx
    d_af = after_idx - before_idx

    th_on, m_on = otsu_mask(d_on, min_abs=min_abs)
    th_af, m_af = otsu_mask(d_af, min_abs=min_abs)

    m_on = cleanup_mask(m_on, min_area=min_area) if m_on is not None else None
    m_af = cleanup_mask(m_af, min_area=min_area) if m_af is not None else None

    if m_on is None or m_af is None:
        return d_on, d_af, None, None, None, (th_on, th_af)

    persistent = m_on & m_af
    transient = m_on & (~m_af)

    return d_on, d_af, m_on, m_af, (persistent, transient), (th_on, th_af)

def show_masks(change, mask, title):
    plt.figure(figsize=(6,6))
    plt.imshow(change)
    plt.title(title)
```

```

plt.axis("off")
plt.colorbar()
plt.show()

if mask is not None:
    plt.figure(figsize=(6,6))
    plt.imshow(mask)
    plt.title(title + " | mask")
    plt.axis("off")
    plt.show()

for tid in sorted(triplets.keys()):
    idx = indices_for_triplet(tid)

    # NDVI tri-temporal
    d_on, d_af, m_on, m_af, pm, ths = tri_temporal_masks(
        idx["BEFORE"]["ndvi_small"], idx["ON"]["ndvi_small"], idx["AFTER"]["ndvi_small"],
        min_abs=0.03, min_area=250
    )
    print(f"\nTriplet {tid} NDVI thresholds (on, after):", ths)
    show_masks(d_on, m_on, f"{tid} | ΔNDVI (ON - BEFORE)")
    show_masks(d_af, m_af, f"{tid} | ΔNDVI (AFTER - BEFORE)")

    if pm is not None:
        persistent, transient = pm
        plt.figure(figsize=(6,6)); plt.imshow(persistent); plt.title(f"{tid} | NDVI Persistent"); plt.axis("off"); plt.show()
        plt.figure(figsize=(6,6)); plt.imshow(transient); plt.title(f"{tid} | NDVI Transient"); plt.axis("off"); plt.show()

    # NDWI tri-temporal
    d_on, d_af, m_on, m_af, pm, ths = tri_temporal_masks(
        idx["BEFORE"]["ndwi_small"], idx["ON"]["ndwi_small"], idx["AFTER"]["ndwi_small"],
        min_abs=0.03, min_area=250
    )
    print(f"Triplet {tid} NDWI thresholds (on, after):", ths)
    show_masks(d_on, m_on, f"{tid} | ΔNDWI (ON - BEFORE)")
    show_masks(d_af, m_af, f"{tid} | ΔNDWI (AFTER - BEFORE)")

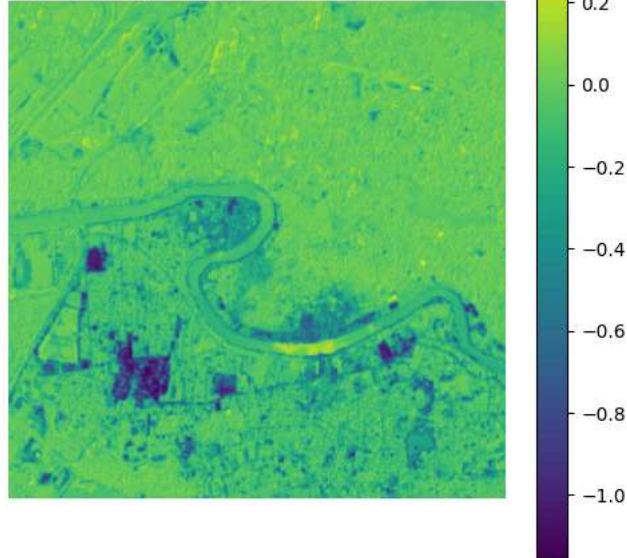
    if pm is not None:
        persistent, transient = pm
        plt.figure(figsize=(6,6)); plt.imshow(persistent); plt.title(f"{tid} | NDWI Persistent"); plt.axis("off"); plt.show()
        plt.figure(figsize=(6,6)); plt.imshow(transient); plt.title(f"{tid} | NDWI Transient"); plt.axis("off"); plt.show()

```

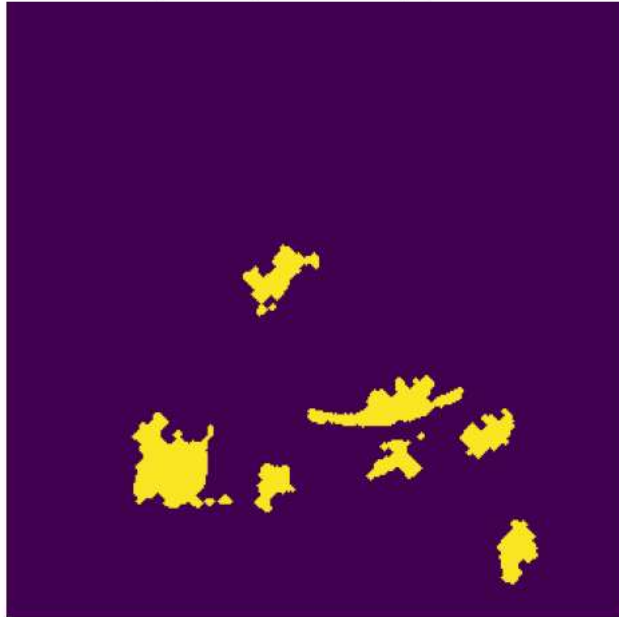


```
Triplet 1 NDVI thresholds (on, after): (np.float32(0.3223071), np.float32(0.24178901))
```

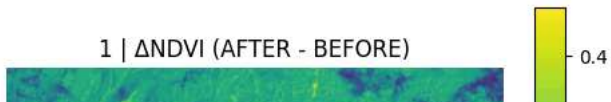
1 | Δ NDVI (ON - BEFORE)

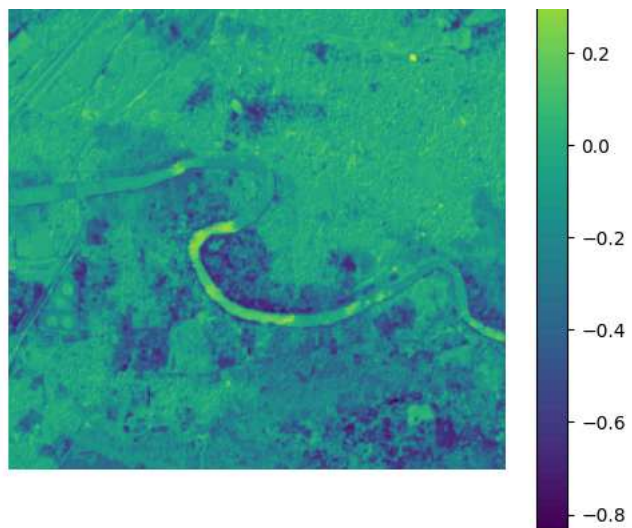


1 | Δ NDVI (ON - BEFORE) | mask



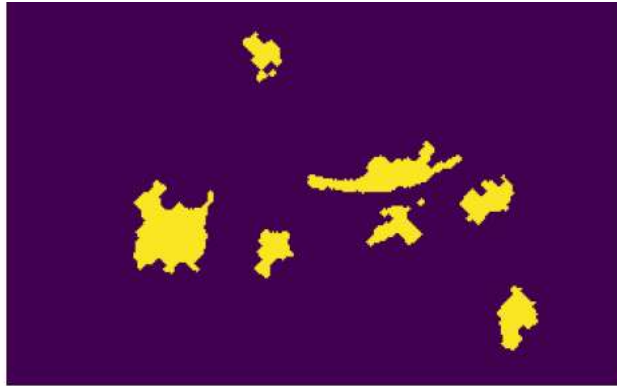
1 | Δ NDVI (AFTER - BEFORE)



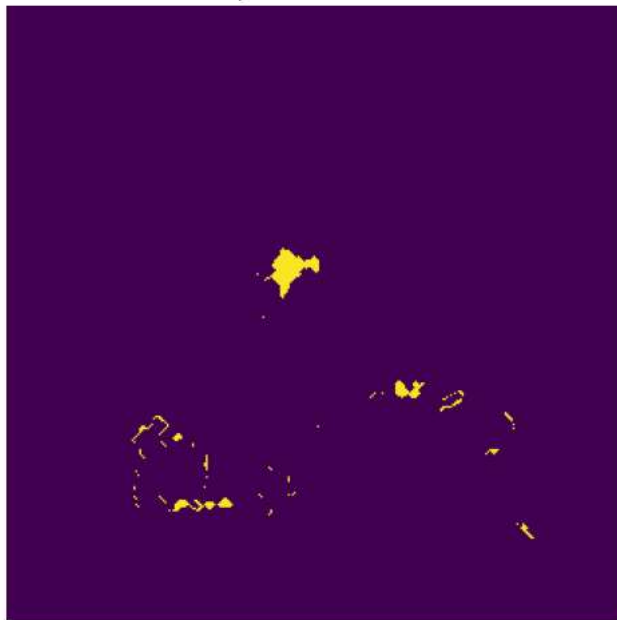
1 | Δ NDVI (AFTER - BEFORE) | mask

1 | NDVI Persistent

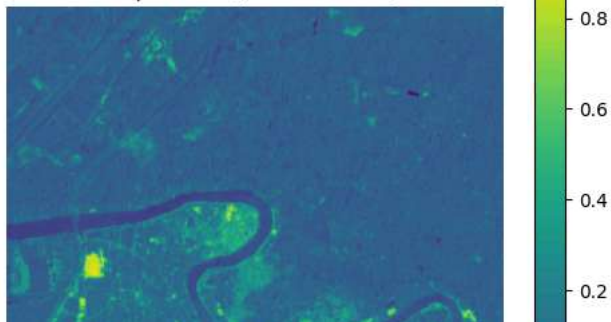


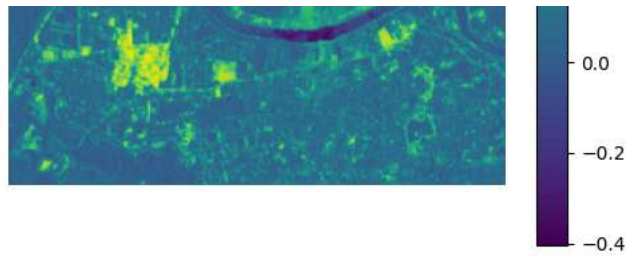
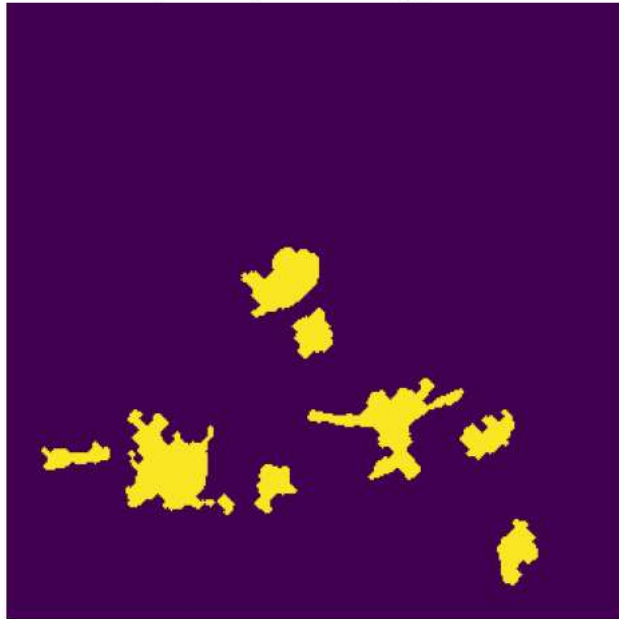
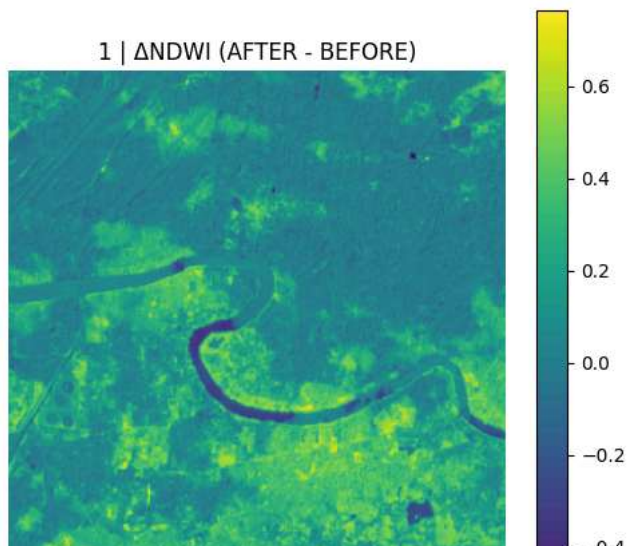


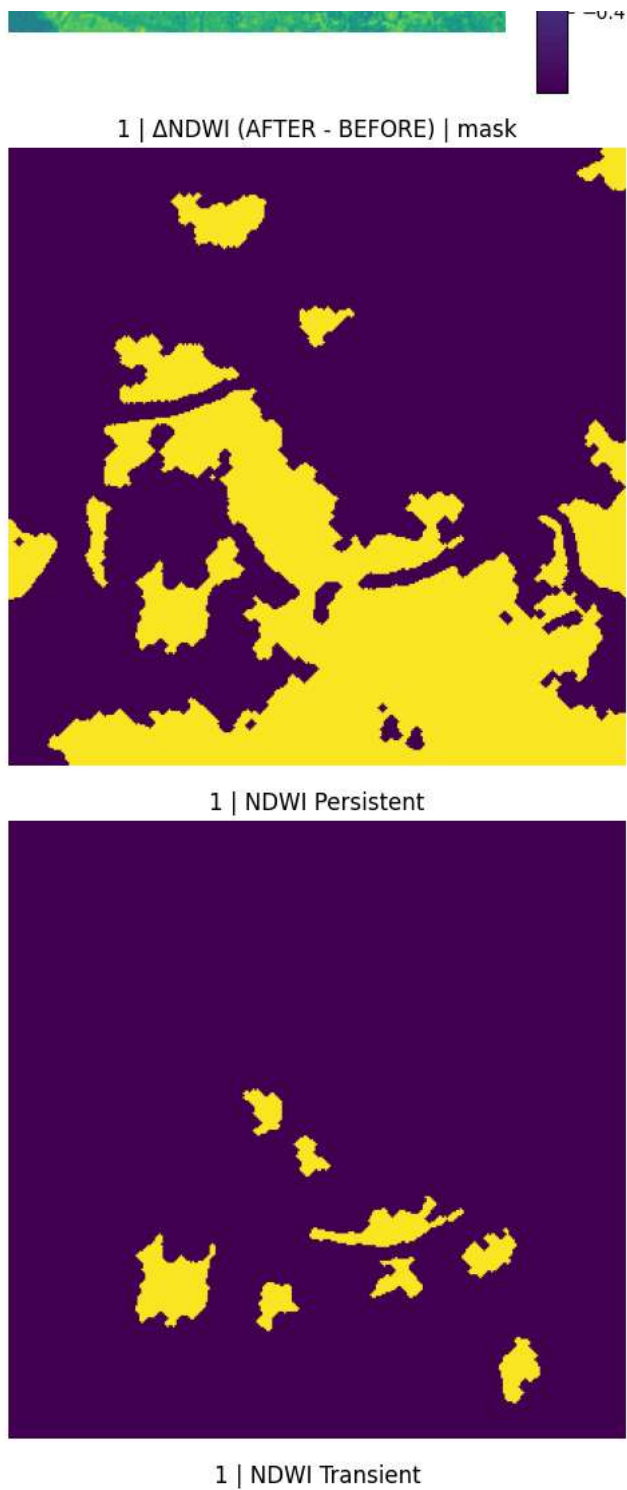
1 | NDVI Transient



Triplet 1 NDWI thresholds (on, after): (np.float32(0.2675923), np.float32(0.21511897))

1 | Δ NDWI (ON - BEFORE)

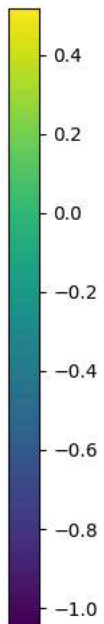
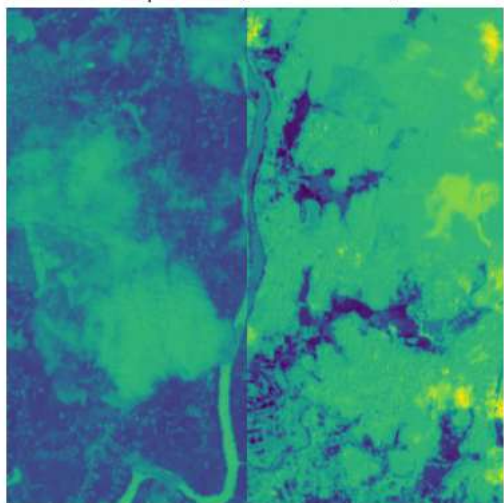
1 | $\Delta\text{NDWI (ON - BEFORE)}$ | mask1 | $\Delta\text{NDWI (AFTER - BEFORE)}$ 





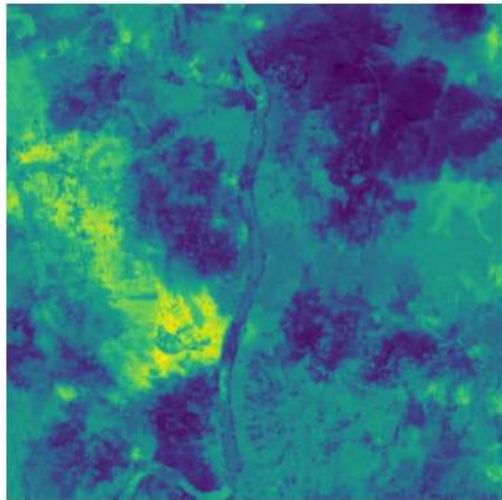
Triplet 15 NDVI thresholds (on, after): (np.float32(0.36971915), np.float32(0.3559786))

15 | Δ NDVI (ON - BEFORE)



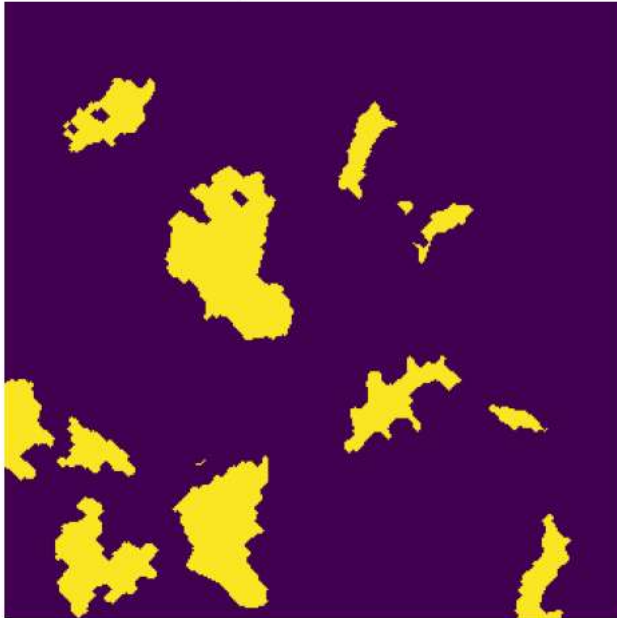
15 | Δ NDVI (ON - BEFORE) | mask



15 | Δ NDVI (AFTER - BEFORE)15 | Δ NDVI (AFTER - BEFORE) | mask



15 | NDVI Persistent



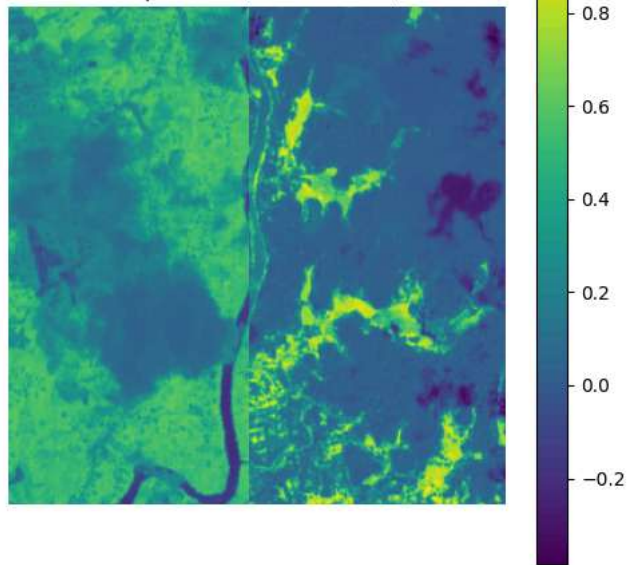
15 | NDVI Transient





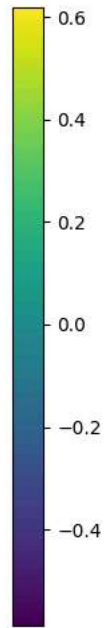
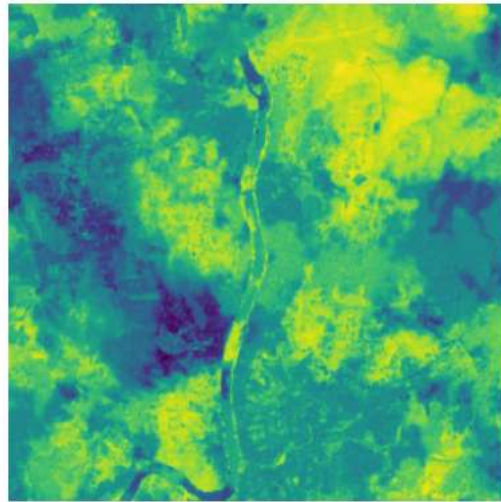
Triplet 15 NDWI thresholds (on, after): (np.float32(0.32077527), np.float32(0.27250737))

15 | Δ NDWI (ON - BEFORE)



15 | Δ NDWI (ON - BEFORE) | mask



15 | Δ NDWI (AFTER - BEFORE)15 | Δ NDWI (AFTER - BEFORE) | mask

15 | NDWI Persistent

